

Typst documenten zijn ook heel goed voor “code-heavy” teksten maar ik denk dat voor een samenvatting of iets dergelijks je ook iets als `TODO: RAW CODE BLOCKS` moet overwegen. Het is op een scherm makkelijker te lezen en heeft bovenal betere syntax highlighting.

Kijk misschien eens naar de HTML features van Typst, aangezien dit mogelijks de perfecte oplossing is. Typst biedt namelijk veel meer dan een simpele markdown file, zonder veel complexiteit toe te voegen.

1 Python `TODO` HEADING STYLING

1.1 Sorting algorithms

1.1.1 Quicksort

Quicksort is een verdeel-en-heers sorteeralgoritme. Het kiest een “pivot”, verdeelt de array zodat kleinere elementen links en grotere rechts komen, en sorteert daarna elke partitie recursief. Gemiddelde tijdscomplexiteit is $O(n \log n)$, worst case $O(n^2)$.

Een simpele implementatie in Python:

```
def quicksort(arr):
    if len(arr) <= 1:
        return arr

    pivot = arr[len(arr) // 2]
    left = [x for x in arr if x < pivot]
    mid = [x for x in arr if x == pivot]
    right = [x for x in arr if x > pivot]

    return quicksort(left) + mid + quicksort(right)
```

Deze versie is zeer overzichtelijk maar gebruikt extra geheugen voor de *sublists*. Een meer geheugen vriendelijke versie sorteert binnen de “array” zelf.

```
def quicksort(arr, low=0, high=None):
    if high is None:
        high = len(arr) - 1
    if low < high:
        p = partition(arr, low, high)
        quicksort(arr, low, p - 1)
        quicksort(arr, p + 1, high)
    return arr

def partition(arr, low, high):
    pivot = arr[high]
    i = low - 1
    for j in range(low, high):
        if arr[j] <= pivot:
            i += 1
            arr[i], arr[j] = arr[j], arr[i]
    arr[i + 1], arr[high] = arr[high], arr[i + 1]
    return i + 1
```

Gebruik in de praktijk altijd deze tweede versie. De eerste is handig om het concept van quicksort te begrijpen.

1.2 Object Oriented Programming

Een *klasse* is een verzameling van *objecten*. Klassen zijn blauwdrukken voor het maken van objecten. Een klasse definieert een verzameling attributen en methoden dat de gecreëerde objecten (*instanties*) kunnen bezitten.

Code

```
class Dog:
    species = "Canine"

    def __init__(self, name, age):
        self.name = name
        self.age = age

dog = Dog("A", 3)
print(dog.name)
print(dog.species)
```

Output

```
>>>python %
A
Canine
```

2 Java **TODO HEADING STYLING**

2.1 Quicksort

```
public class QuickSort {
    public static void quickSort(int[] arr, int low, int high) {
        if (low < high) {
            int p = partition(arr, low, high);
            quickSort(arr, low, p - 1);
            quickSort(arr, p + 1, high);
        }
    }

    private static int partition(int[] arr, int low, int high) {
        int pivot = arr[high];
        int i = low - 1;
        for (int j = low; j < high; j++) {
            if (arr[j] <= pivot) {
                i++;
                int temp = arr[i];
                arr[i] = arr[j];
                arr[j] = temp;
            }
        }
        int temp = arr[i + 1];
        arr[i + 1] = arr[high];
        arr[high] = temp;
        return i + 1;
    }

    public static void main(String[] args) {
```

```
int[] arr = {3, 6, 1, 8, 2, 9, 4};  
quickSort(arr, 0, arr.length - 1);  
System.out.println(java.util.Arrays.toString(arr));  
}  
}
```